

UD1.EXAME Práctico

DAM1-Programación 2026-27

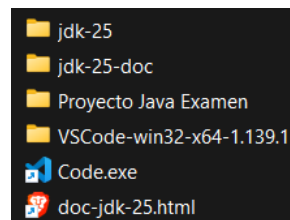
2026/10/05

1. Faltas serializadas (3)	2
2. Leer Empleados.xml (3)	2
3. Exportar Json de empleados(2)	3
4. Meteogalicia Temperatura Concello (2)	4
Orientación de Evaluación	4

- **Apaga y guarda tu teléfono móvil. Coloca tu mochila y abrigo junto a la entrada para recogerlos al terminar y salir.**
- Realiza los programas **individualmente y en silencio**. En caso contrario se retirará el examen y se calificará con un cero.
- Si necesitas **consultar al profesor** llama su atención levantando la mano y espera a que se acerque.
- **Cuida el código fuente** y realiza programas bien estructurados, legibles, con comentarios, líneas en blanco, identificadores adecuados, etc.
- **Cuida la interacción con el usuario**, presentando la información de forma clara y ordenada, sin faltas de ortografía.
- **Utiliza los casos de prueba** de ejemplo si los hay para probar tu programa y añade tus propios casos.
- NO se valorarán programas con **errores de compilación**.
- NO se permite el uso de sentencias, estructuras o **técnicas no vistas en clase**. El profesor podrá solicitar **explicaciones adicionales** sobre el código entregado.
- Utiliza los **últimos minutos** de examen para verificar que tu entrega es correcta.

Entorno de Programación proporcionado:

- IDE Visual Studio Code con Java Extensions Pack
- JDK 25 con documentación de la API.
- Proyecto Java para realizar el examen.
- Librerías y dependencias necesarias.
- Apuntes y material didáctico adicional en PDF.



1. Crea un **nuevo proyecto Java** para realizar el examen, o utiliza el proporcionado.
2. Crea la siguiente estructura de **paquetes** bajo la carpeta **src**: **ud1.examen.xxx** donde **xxx** es tu **nombre de usuario en el sistema del centro**.
3. Crea dentro del paquete **xxx** los ficheros **.java** de cada ejercicio con el nombre apropiado.
4. Incluye en cada programa un **comentario de documentación de autoría** con tu nombre.
5. **Entrega la carpeta xxx comprimida en la tarea del Aula Virtual.**
Desde el explorador de archivos de Windows haz clic con el botón derecho sobre la carpeta **xxx** y elige "Comprimir en .zip". Entrega el fichero resultante.
6. **Tiempo estimado: 2 horas**

1. Faltas serializadas (3)

FaltasSerializadas.java

El fichero **DATOS/Faltas.csv** contiene una relación de faltas de alumnado en un centro educativo con el siguiente formato de ejemplo:

```
"Identificador";"Nome completo";"Curso (abrev.)";"Dt. falta";"Tipo falta";"Materia ou módulo";"Sesión";"Xustificada";  
"00000000A";"Apelidos, Nome1";"ADSIINRE";"18/09/2026";"Asistencia";"Itinerario persoal para a empregabilidade II";"6";"Non";
```

De dicho fichero se desean extraer las faltas de asistencia no justificadas y guardarlas en un fichero binario **DATOS/FaltasAsistencia.dat** de objetos serializados de la siguiente clase:

```
public class FaltaAsistencia {  
    LocalDate data;  
    int sesion;  
    String modulo;  
    String curso;  
    String Alumno;
```

Completa la clase **FaltaAsistencia** y elabora un programa que genere el fichero binario.

2. Leer Empleados.xml (3)

LeerEmpleadosXml.java

Dado el siguiente fichero **DATOS\Empleados.xml**:

```
<?xml version="1.0" encoding="UTF-8"?>  
<empleados>  
  <empleado id="1">  
    <nombre>Ana García</nombre>  
    <departamento>Informática</departamento>  
    <salario>35000</salario>  
  </empleado>  
  <empleado id="2">  
    <nombre>Luis Pérez</nombre>  
    <departamento>Recursos Humanos</departamento>  
    <salario>28000</salario>  
  </empleado>  
  <empleado id="3">  
    <nombre>María López</nombre>  
    <departamento>Informática</departamento>  
    <salario>42000</salario>  
  </empleado>  
</empleados>
```

```
public class Empleado {  
    int id;  
    String nombre;  
    String departamento;  
    double salario;  
}
```

Completa la clase Empleado e implementa el siguiente método que leerá el contenido del fichero y devolverá una lista de empleados:

```
public static List<Empleado> leerEmpleados(String fichero)
```

Usando el método anterior elabora un programa principal que imprima por pantalla el listado de empleados del fichero XML proporcionado en el formato del método toString() generado por el IDE.

```
Empleado [id=1, nombre=Ana García, departamento=Informática, salario=35000.0]  
Empleado [id=2, nombre=Luis Pérez, departamento=Recursos Humanos, salario=28000.0]  
Empleado [id=3, nombre=María López, departamento=Informática, salario=42000.0]
```

3. Exportar Json de empleados(2)

ExportarJsonEmpleados.java

Elabora un método que genere un fichero en formato JSON con el contenido de una lista de empleados correspondientes a la clase del ejercicio anterior.

```
public static boolean exportarJson(List<Empleado> empleados, String fichero)
```

Si por algún motivo no es posible realizar la exportación el método devolverá false. En caso de éxito devolverá true.

Elabora un programa principal que utilice el método anterior para exportar la siguiente lista de empleados al fichero empleados.json:

```
empleados = List.of(  
    new Empleado(1, "Ana García", "Informática", 35000),  
    new Empleado(2, "Luis Pérez", "Recursos Humanos", 28000),  
    new Empleado(3, "María López", "Informática", 42000));
```

4. Meteogalicia Temperatura Concello (2)

MeteogaliciaTemperaturaConcello.java

O ficheiro **DATOS/observacionConcellos.json** contén o resultado de consultar a seguinte URL de Meteogalicia:

- https://servizos.meteogalicia.gal/mgrss/observacion/observacionConcellos.action?request_locale=gl

Elabora o seguinte método que recibe como entrada o contido en formato JSON e o nome dun concello e devolve a temperatura rexistrada:

```
public static Double temperaturaConcello(String jsonObservacion, String concello)
```

Se se produce calquera erro o método informará da excepción correspondente na Saída de erros estándar e devolverá **null**.

Crea un programa que utilice o método anterior e amose a temperatura dun concello, por exemplo “Marín”, segundo os datos do ficheiro JSON.

Orientación de Evaluación

- Cada exercicio puntúa sobre a nota numérica indicada entre paréntesis.
- Para cada exercicio, a modo de orientación se valorará:
 - **Entrega:** identificadores de paquete y clases, comentario de autor.
 - **Buenas prácticas de programación:** elección de tipos e identificadores, legibilidade, regras de estilo Java, mensaxes claros en la interacción con el usuario, etc.
 - **Funcionalidad:** que el programa haga lo que se pide con exactitud.
 - El manejo adecuado de excepciones.
 - La correcta organización de las clases.
 - Cierre adecuado de ficheros y flujos.